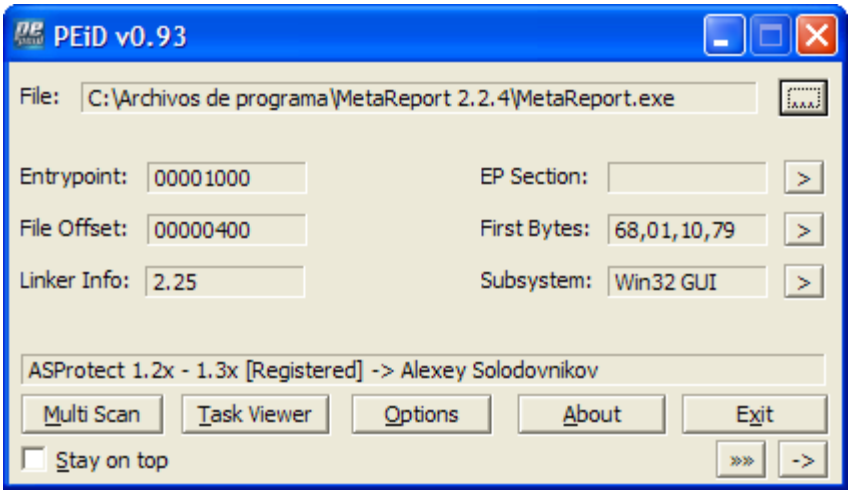
Estudio de ASProtect 1.2x - 1.3x [Registered]

Programa:	MetaReport 2.2.4
Protección:	Empaquetado con ASProtect
Objetivos:	Desempakar y conseguir que funke
URL descarga:	http://www.metamatica.com
Dificultad:	Bastante sencillo
Herramientas:	OllyDbg v1.10 + PEiD v0.93 + ImpREC v1.6 + ConTEXT v0.98.3 + RDG packer v0.6.3 Beta (opcional)
Cracker:	SyXe'05 .::[hAcK-c0d3d]..[CracksLatinoS]::. Fecha: 05-02-06

Un saludo a todo CracksLatinoS!

Muy buenas compañeros, vamos escribir un tutorial de uno de nuestros packers preferidos y más conocidos: ASProtect en una de sus versiones "intermedias" ni muy antigua ni muy nueva (No tiene stolen code) un tema que intentaremos desvelar más adelante. Por ahora nos concentraremos en este objetivo, se llama MetaReport v2.2.4, útil para hacer informes y tal aunque personalmente no le encuentro utilidad, poco importa, lo analizamos con PEiD v0.94 y nos dice esto:



Hay veces que PEiD nos muestra esta versión cuando puede ser otra, veamos:

Eso si lo analizamos en modo M-A pues con el M-B nos dice ASPack v2.12 que como ahora veremos no es así.

Cerramos ambos analizadores y abrimos el ejecutable MetaReport.exe en un Olly, yo utilizo la versión 1.10 con muy pocos plugins:

1.- CommandBar v3.10.109c by Oleg Yuschuk (creador de Olly)
2.- Hide Debugger v1.2.3f by Asterix
3.- OllyDump v2.21.108 by Gigapede
4.- OllyScript v0.92 by SHaG

Son realmente pocos, a veces utilizo el HideOD que se utiliza para ocultar el debugger pero con muchas más opciones pero no es este el caso, pues sabemos que este packer únicamente tiene una protección anti-debug (byte de detección IsDebuggerPresent) que lee mediante una llamada a esa api, para esto no será necesario ningún plugin más (por el momento) si luego nos hiciera falta pues se lo agregamos sobre la marcha.

Y ahora vamos a ver cómo trabaja esta versión del packer, ya que no todos trabajan igual ni hacen lo mismo, por un lado a causa de las distintas versiones existentes y por otro las diferentes opciones de empakado que poseen cada una de ellas. Lo que haremos será un breve repaso bajo Olly del .exe protegido a ver cuál es el esquema a derrotar.

Vamos a FILE - OPEN y abrimos MetaReport.exe, quedamos detenidos en su EP:

```
Address   Hex dump        Disassembly
00401000  68 01107900     push    00791001
00401005  E8 01000000     call    0040100B
0040100A  C3              ret
0040100B  C3              ret
0040100C  09BA 53B3FDE1   or      [edx+E1FDB353],edi
00401012  34 66           xor     al,66
00401014  0129            add     [ecx],ebp
00401016  0B4D 54         or      ecx,[ebp+54]
```

Ese inicio nos es familiar XDD. Destildamos todas las excepciones para que se detenga en ellas desde OPTIONS - DEBUGGING OPTIONS - EXCEPTIONS para que Olly se vaya deteniendo en ellas, vamos a utilizar el fantástico método de Ricardo para descubrir el OEP (método de las excepciones), la última es ésta:

```
Address   Hex dump     Disassembly                    Comment
014FF59B  0156 00      add     [esi],edx
014FF59E  825A 52 F2   sbb     byte ptr [edx+52],-0E
014FF5A2  A4           movsb
014FF5A3  D267 64      shl     byte ptr [edi+64],cl
014FF5A6  8F06         pop     dword ptr [esi]
014FF5A8  0000         add     [eax],al
014FF5AA  83C4 04      add     esp,4
014FF5AD  EB 01        jmp     short 014FF5B0
014FF5AF  F3:          prefix rep:                    Superfluous prefix
014FF5B0  C1CE DD      ror     esi,0DD                Shift constant out
```

Si la pasamos con SHIFT+F9 el programa arranca, entonces reiniciamos Olly con CTRL+F2 y llegamos a ella de nuevo, pero ahora antes de pasarla ponemos un BPM en la sección de código que suele ser la siguiente al PE HEADER la pasamos y..

```
Address   Hex dump     Disassembly                    Comment
00631BF8  55           push    ebp          <-- OEP¡!!!
00631BF9  8BEC         mov     ebp,esp
00631BFB  83C4 F0      add     esp,-10
00631BFE  53           push    ebx
00631BFF  B8 F0126300  mov     eax,006312F0
00631C04  E8 3B59DDFF  call    00407544
00631C09  E8 0AEDFFFF  call    00630918
00631C0E  84C0         test    al,al
00631C10  75 22        jnz     short 00631C34
00631C12  A1 44876300  mov     eax,[638744]
00631C17  8B00         mov     eax,[eax]
00631C19  E8 1AB3E7FF  call    004ACF38
00631C1E  8B1D 54816300 mov    ebx,[638154]           MetaRepo.00637EF8
00631C24  8B1B         mov     ebx,[ebx]
00631C26  FFD3         call    ebx
```

Paró en el OEP del programa, es decir, el Entry Point que tenía antes de ser protegido con ASProtect, este dato por supuesto lo anotamos para luego:

OEP==00631BF8

Ahora le echamos un vistazo al código a ver dónde están las maldades de este packer, las más relevantes son éstas:

```
Address   Hex dump        Disassembly                      Comment
00401328  - FF25 3C8B5101 jmp     [1518B3C]                kernel32.CloseHandle
0040132E    8BC0          mov     eax,eax
00401330  - FF25 488B5101 jmp     [1518B48]                kernel32.CreateFileA
00401336    8BC0          mov     eax,eax
00401338    E8 C3EC3401   call    01750000  <---- Tenemos problemas :(
0040133D    14 8B         adc     al,8B
0040133F    C0E8 BB       shr     al,0BB                   Shift constant out of
00401342    EC            in      al,dx                    I/O command
00401343    36:0188 8BC0E8B3 add  ss:[eax+B3E8C08B],ecx
0040134A    EC            in      al,dx                    I/O command
0040134B    3801          cmp     [ecx],al
0040134D  ^ 79 8B         jns     short 004012DA
0040134F    C0FF 25       sar     bh,25                    Shift constant out of r
00401352  ^ 78 8B         js      short 004012DF
00401354    51            push    ecx
```

Como se ve la JMP Table está totalmente destrozada, la mayoría de saltos a la iat tipo JMP DWORD PTR [POS_IAT] fueron reemplazados por CALL ASPROTECT lo que significa que aún dumpeando el archivo no funcionará porque las llamadas a las funciones importadas están redirigidas a una sección del packer que en nuestro dumpeado no existirá. Aquí está el fuerte de la protección. Además si nos fijamos hay una especie de IAT Scrambled como en el Armadillo, la sección a la que apuntan los JMP están en una sección temporal:

Address	Size	Owner	Section	Contains	Type	Access	Initial
014C0000	00001000				Priv	RWE	RWE
014D0000	00001000				Priv	RWE	RWE
014E0000	0002B000				Priv	RWE	RWE
01510000	00004000				Priv	RW	
01518000	00004000				Priv	RW	
01610000	00051000				Map	R	R
01670000	00001000				Priv	RWE	RWE

Y los CALL apuntan a otra muy distintas como esta:

```
Address   Hex dump        Disassembly                      Comment
00401330  - FF25 488B5101 jmp     [1518B48]                kernel32.CreateFileA
00401336    8BC0          mov     eax,eax
00401338    E8 C3EC3401   call    01750000
0040133D    14 8B         adc     al,8B
0040133F    C0E8 BB       shr     al,0BB                   Shift constant out o
00401342    EC            in      al,dx                    I/O command
00401343    36:0188 8BC0E8B3 add  ss:[eax+B3E8C08B],ecx
0040134A    EC            in      al,dx                    I/O command
```

Ese CALL apunta a otra sección temporal, si nos desplazamos hasta allí con GOTO - EXPRESSION veremos el código que ejecutará esta entrada, aunque es más sencillo pulsar ENTER estando sobre el CALL, de cualquier manera veremos esto:

```
Address   Hex dump        Disassembly                      Comment
01750000    8D87 1C1C4700 lea     eax,[edi+471C1C]
01750006    65:EB 01      jmp     short 0175000A           Superfluous prefix
01750009    9A EB01F203 4424 call far 2444:03F201EB        Far call
01750010    1803          sbb     [ebx],al
01750012    44            inc     esp
01750013    24 38         and     al,38
01750015    8D4424 42     lea     eax,[esp+42]
01750019    8D4420 BE     lea     eax,[eax-42]
0175001D    83C0 04       add     eax,4
01750020    8BE0          mov     esp,eax
01750022    68 00007401   push    1740000
01750027    C3            ret
```

Lo más significativo es lo resaltado en la imagen, ejecuta instrucciones hasta llegar a un PUSH - RET que desviará la ejecución hacia :01740000 donde hay esto:

```
Address   Hex dump        Disassembly                    Comment
01740000    55            push    ebp
01740001    8BEC          mov     ebp,esp
01740003    83EC 10       sub     esp,10
01740006    53            push    ebx
01740007    64:A1 18000000 mov    eax,fs:[18]
0174000D    8B5D 08       mov     ebx,[ebp+8]
01740010    83FB F4       cmp     ebx,-0C
01740013    8B40 30       mov     eax,[eax+30]
01740016  - 0F84 2AE2707C ja      kernel32.77E4E246
0174001C    83FB F5       cmp     ebx,-0B
0174001F  v EB 01         jmp     short 01740022
01740021    0F0F          ???                            Unknown command
```

Lo interesante otra vez está resaltado, si seguimos el salto vamos a parar a una zona del kernel donde hay un JMP que nos lleva a:

```
Address                    Hex dump        Disassembly                Comm
77E5E3F8                     C2 0800       ret     8
77E5E3FB kernel32.GetFileType 55           push    ebp
77E5E3FC                     8BEC          mov     ebp,esp
77E5E3FE                     83EC 10       sub     esp,10
77E5E401                     53            push    ebx
77E5E402                     64:A1 18000000 mov    eax,fs:[18]
77E5E408                     8B5D 08       mov     ebx,[ebp+8]
77E5E40B                     83FB F4       cmp     ebx,-0C
77E5E40E                     8B40 30       mov     eax,[eax+30]
77E5E411                   ^ 0F84 2FFEFEFF ja      77E4E246
77E5E417                     83FB F5       cmp     ebx,-0B
77E5E41A                   ^ 0F84 1BFEFEFF ja      77E4E23B
77E5E420                     83FB F6       cmp     ebx,-0A
77E5E423                   v 0F84 2EC50100 ja      77E7A957
77E5E429                     8BC3          mov     eax,ebx
77E5E42B                     25 03000010   and     eax,10000003
```

La línea ROJA indica el destino del salto, si miramos por encima vemos que comienza una api que es GetFileType, pues bien, ahí es donde deberíamos haber saltado directamente en lugar de ir a :01750000, echando un vistazo deducimos lo que hace el packer:

- Intercepta el momento de rellenar la iat.
- Crea una sección temporal donde copia código inerte.
- Crea una segunda sección temporal.
- Al final de la primera sección creada introduce un salto a la segunda zona por medio de un PUSH - RET.
- En la segunda sección copia parte de la api interceptada y un salto a mitad de la misma según corresponda.
- Reemplaza el JMP DWORD PTR [POS_IAT] por un CALL ASPROTECT.

Esto mismo lo hace con las que le parece bien, hay otras que no toca y otras que emula por completo como GetCommandLineA, GetCurrentProcessId, GetVersion, etc.. y un caso especial que es GetProcAddress como ahora veremos.

Echado el vistazo de rigor pasamos a la segunda parte del tute…

Comienza el estudio XDD

Hoy no profundizaremos demasiado en el packer porque no he encontrado nada especial que no se haya comentado en otros tutes de esta protección, veremos los puntos más calientes e importantes para comprender su funcionamiento.

Dejamos este Olly parado en el OEP por si necesitamos mirar algo y abrimos otro Olly, estamos en el EP veamos el primer CALL ASPROTECT que nos ha puesto:

```
00401328  - FF25 3C8B5101   jmp     [1518B3C]        ; kernel32.CloseHandle
0040132E    8BC0            mov     eax,eax
00401330  - FF25 488B5101   jmp     [1518B48]        ; kernel32.CreateFileA
00401336    8BC0            mov     eax,eax
00401338    E8 C3EC3401     call    01750000
0040133D    14 8B           adc     al,8B
```

Aunque también nos valen los JMP anteriores, la cuestión es localizar el lugar donde introduce estos valores, para ello nos fijamos en sus bytes finales:

```
Address   Hex dump                                          ASCII
00401328  FF 25 3C 8B 51 01 8B C0 FF 25 48 8B 51 01 8B C0
00401338  E8 C3 EC 34 01 14 8B C0 E8 BB EC 36 01 88 8B C0
00401348  E8 B3 EC 38 01 79 8B C0 FF 25 78 8B 51 01 8B C0
00401358  FF 25 84 8B 51 01 8B C0 FF 25 90 8B 51 01 8B C0
00401368  E8 93 EC 3B 01 6C 8B C0 FF 25 A8 8B 51 01 8B C0
00401378  FF 25 B4 8B 51 01 8B C0 E8 7B EC 3E 01 C9 8B C0
00401388  FF 25 CC 8B 51 01 8B C0 E8 6B EC 41 01 A1 8B C0
00401398  FF 25 E4 8B 51 01 8B C0 FF 25 F0 8B 51 01 8B C0
004013A8  E8 53 EC 44 01 D5 8B C0 E8 4B EC 46 01 DE 8B C0
```

Esos bytes pertenecen al primer JMP que marca [1518B3C] que es a CloseHandle, no nos importa la api, por ahora sólo buscamos el lugar donde introduce los bytes que conforman el salto. Esto lo conseguimos seleccionando el primer byte del mismo a partir del 0xFF25 que despreciamos ya que no influye siempre es igual, marcaremos el byte 0x3Ch de :40232A que este sí forma parte de los bytes buscados, le ponemos un BPM ON MEMORY - WRITE ya que ASProtect no para en los HBPs porque los borra en el SEH. Para ir más rápidos podemos deshabilitar las excepciones temporalmente marcando la casilla:

☑ Ignore memory access violations in KERNEL32

Debemos tener además las excepciones agregadas allí sino no servirá de nada, si no las tenemos las podemos ir agregando con ADD LAST EXCEPTION, cuando lo tengamos bien configurado apretamos F9 y Olly para en un loop:

```
Address   Hex dump   Disassembly                          Comment
014E996A  881C3E     mov     [esi+edi],bl                 // desencripta codigo
014E996D  FF45 F0    inc     dword ptr [ebp-10]
014E9970  FF4D EC    dec     dword ptr [ebp-14]
014E9973  75 95      jnz     short 014E990A
014E9975  5F         pop     edi
014E9976  5E         pop     esi
014E9977  5B         pop     ebx
014E9978  8BE5       mov     esp,ebp
014E997A  5D         pop     ebp
014E997B  C2 9400    ret     94
```

Está desencriptando código pero todavía es muy pronto, volvemos a pulsar F9 para que corra y nos para en un REP MOVS que tampoco interesa porque escribió otros bytes que no son los finales, al final los pone aquí:

```
Address   Hex dump   Disassembly                          Comment
014F5E4F  8906       mov     [esi],eax                    // escribe los bytes
014F5E51  0FB74424 0 movzx   eax,word ptr [esp+4]
014F5E56  0143 08    add     [ebx+8],eax
014F5E59  E9 B1FCFFF jmp     014F5B0F                     al inicio del loop..
014F5E5E  3C 03      cmp     al,3
014F5E60  0F85 A500  jnz     014F5F0B
014F5E66  EB 01      jmp     short 014F5E69
014F5E68  698B 43088 imul    ecx,[ebx+8A0843],FE084
014F5E72  C8 7409FE  enter   974,0FE
014F5E76  C8 7441E9  enter   4174,0E9
```

Aquí paró, no debemos olvidar ir quitando los BPs que pongamos pues verifica el código mediante rutinas CRC32, si miramos los registros ahora veremos:

```
Registers (FPU)
EAX 01518B3C <------ bytes finales
ECX 77F432FA ntdll.77F432FA
EDX 77FB306C ntdll.77FB306C
EBX 015103E0
ESP 0012FEF8
EBP 77E5A6F0 kernel32.CloseHandle
ESI 0040132A MetaRepo.0040132A
EDI 01518B3C
```

Ahí está todo lo necesario, tenemos en EAX los bytes buscados, en EBP la api correcta y en ESI la dirección de la JMP Table donde colocará el salto (JMP) no está nada mal, jeje.. si nos fijamos un poco más abajo de la zona donde guarda los bytes hay un JMP 014F5B0F que nos lleva al posible punto gatillo

pues allí se dirige cada vez que inserta un JMP o CALL en la tabla de saltos, esto hay que tenerlo en cuenta para un posible script.

Ahora de momento estudiaremos un poco lo que va haciendo, ponemos un BPM ON MEMORY - WRITE en la zona grisada que es parte de la tabla de saltos:

```
Address   Hex dump                                          ASCII
0040132A  F1 EC D3 8F 8B C0 FF 25 BA 68 E0 0C 8B C0 00 00  ñì Ó¤‹À ÿ%ºhà.‹À..
0040133A  A7 77 3A 14 8B C0 00 00 48 F6 16 88 8B C0 00 00  §w:¶‹À..Ho▬^‹À..
0040134A  6D 73 B8 79 8B C0 FF 25 26 F9 8E 87 8B C0 FF 25  ms¸y‹À ÿ%&ùŽ‡‹À ÿ
0040135A  C3 F9 7A 5F 8B C0 FF 25 D4 F0 4F 34 8B C0 00 00  Ãùz_‹À ÿ%ÔðO4‹À..
0040136A  29 88 65 6C 8B C0 FF 25 D2 50 81 09 8B C0 FF 25  )ˆel‹À ÿ%ÒP▯.‹À ÿ
0040137A  1F 60 CB FB 8B C0 00 00 A0 60 89 C9 8B C0 FF 25  ▼`ËÛ‹À.. `‰É‹À ÿ%
0040138A  25 E7 89 D2 8B C0 00 00 BE 1B 11 A1 8B C0 FF 25  %ç‰Ò‹À..¾←◄¡‹À ÿ%
0040139A  BB 06 9C CD 8B C0 FF 25 AC 11 65 8D 8B C0 00 00  »♠œÍ‹À ÿ%¬◄eÍ‹À..
0040140A  61 8C 7E D5 8B C0 00 00 EA 45 EB DE 8B C0 00 00  aŒ~Õ‹À..êEëÞ‹À..
0040141A  97 89 FD 31 8B C0 00 00 F8 0F A6 C1 8B C0 00 00  —‰ý1‹À..ø☼¦Á‹À..
0040142A  DD B3 42 B0 8B C0 00 00 56 FB FB 64 8B C0 00 00  Ý³B°‹À..VûÛd‹À..
0040143A  B3 C4 3C 79 8B C0 00 00 84 A7 3C 79 8B C0 00 00  ³Ä<y‹À..„§<y‹À..
0040144A  99 D9 B7 08 8B C0 00 00 02 A8 9A 94 8B C0 FF 25  ™Ù·◘‹À..☺ ¨šö”‹À ÿ%
0040145A  0F D4 BE FD 8B C0 00 00 50 64 92 7A 8B C0 00 00  ☼Ô¾ý‹À..Pd’z‹À..
0040146A  95 B9 1D 8B 8B C0 00 00 EE F7 F5 87 8B C0 FF 25  •¹↔‹À..î÷õ‡‹À ÿ%
0040147A  AB 13 1D B3 8B C0 FF 25 5C 12 16 7B 8B C0 FF 25  «‼↔³‹À ÿ%\↕▬{‹À ÿ%
0040142A  D1 4F CE 43 8B C0 FF 25 1A D7 BF C2 8B C0 00 00  ÑOÎC‹À ÿ%→×¿Â‹À..
0040143A  87 1F 81 49 8B C0 00 00 A8 BD 07 40 8B C0 00 00  ‡▼ü‹À.. ¨½•@‹À..
0040144A  4D D8 39 73 8B C0 FF 25 86 71 19 46 8B C0 FF 25  MØ9s‹À ÿ%†q↓F‹À ÿ%
0040145A  A3 D3 40 41 8B C0 00 00 34 B2 84 70 8B C0 FF 25  £Ó@A‹À..4²„p‹À ÿ%
0040146A  09 CF 22 D8 8B C0 FF 25 32 33 BF F3 8B C0 FF 25  .Ï"Ø‹À ÿ%23¿ó‹À ÿ%
0040147A  FF 4B BA 8A 8B C0 FF 25 00 7C D3 24 8B C0 FF 25  ÿKºŠ‹À ÿ%.|Ó$‹À ÿ%
0040148A  05 F0 19 AC 8B C0 00 00 1E C8 74 6C 8B C0 00 00  ♣ð↓¬‹À..▲Ètl‹À..
0040149A  9B E4 6F 8E 8B C0 FF 25 0C E7 EF 7E 8B C0 FF 25  ›äoŽ‹À ÿ%♀çï~‹À ÿ%
004014AA  41 37 3A CE 8B C0 53 83 C4 BC BB 0A 00 00 00 54  A7:Î‹ASƒÄ¼»....T
004014BA  E8 41 FF FF FF F6 44 24 2C 01 74 05 0F B7 5C 24  èAÿÿÿöD$,.t♪.·\$
```

Para ver donde va parando y lo que va hace tomando nota de ello, pulsamos F9 y tras varias paradas en el mismo punto hace una parada aquí:

```
Address   Hex dump   Disassembly                    Comment
014F3AEE    C603 E8   mov     byte ptr [ebx],0E8   // opcode de CALL
014F3AF1    43        inc     ebx
014F3AF2    8903      mov     [ebx],eax
014F3AF4    5B        pop     ebx
014F3AF5    C3        ret
014F3AF6    8BC0      mov     eax,eax
014F3AF8    8A10      mov     dl,[eax]
014F3AFA    40        inc     eax
014F3AFB    8A00      mov     al,[eax]
```

Acabamos de encontrar la zona donde inserta los CALL ASPROTECT, veamos que valores hay en los registros en este momento:

```
Registers (FPU)
EAX 0134ECC3  <---- bytes del call
ECX 0012FF01
EDX 0134ECC3
EBX 00401338 MetaRepo.00401338
ESP 0012FEF0
EBP 77E5E3FB kernel32.GetFileType
ESI 0040133A MetaRepo.0040133A
EDI 01[illegible]
```

Tenemos en EAX los bytes del call, en EBP la api correcta y en ESI la dirección donde irá dicho call, datos importantes a tener en cuenta, si seguimos pulsando F9 descubrimos una tercera zona distinta:

```
Address   Hex dump   Disassembly                    Comment
014F5D53    8906      mov     [esi],eax
014F5D55    0FB74424 0 movzx   eax,word ptr [esp+4]
014F5D5A    0143 08   add     [ebx+8],eax
014F5D5D  ^ E9 ADFDFFF jmp    014F5B0F
014F5D62    3C 04     cmp     al,4
014F5D64  v 0F85 F400 jnz     014F5E5E
014F5D6A    EB 01     jmp     sh[illegible] 014F[illegible]6D
```

Veamos los registros:

```
Registers (FPU)
EAX 01518B90  <---- bytes del JMP
ECX 00000000
EDX 01518B90
EBX 015103E0
ESP 0012FEF8
EBP 77F50C44 ntdll.RtlUnwind
ESI 00401362 MetaRepo.00401362
EDI 01518B00
```

En EAX están los bytes pertenecientes al salto tratado de :401362 como marca ESI, después en EBP está la api correcta a la que debería apuntar, pero este no es el último punto donde salva valores pues hay más:

```
Address   Hex dump     Disassembly                    Comment
014F5EA2   8906        mov     [esi],eax
014F5EA4   8A4C24 06   mov     cl,[esp+6]
014F5EA8   8B5424 14   mov     edx,[esp+14]
014F5EAC   8B4424 0C   mov     eax,[esp+C]
014F5EB0   E8 FFF0FFF  call    014F4FB4
014F5EB5 ^ E9 55FCFFF  jmp     014F5B0F
014F5EBA   8BC3        mov     eax,ebx
```

Y los registros en este momento marcan:

```
Registers (FPU)
EAX 01518C74 <---- bytes del JMP
ECX 01518C74
EDX 00000000
EBX 015103E0
ESP 0012FEF8
EBP 77E5AD86 kernel32.GetModuleHandleA
ESI 004013FA MetaRepo.004013FA
EDI 01518C00
```

En este caso se trata de GetModuleHandleA, pero en lugar de la api colocará un salto indirecto donde apunta [01518C74] como marca EAX en :4013F8, en un principio podemos creer que irá a GetModuleHandleA pero si vamos allí vemos:

```
Address   Hex dump                                          ASCII
01518C74  B8 46 4F 01 00 00 00 00 0E 00 00 00 00 00 99 01  .FO.....
01518C84  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
01518C94  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
01518CA4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
01518CB4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
01518CC4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
01518CD4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ........
```

Si ahora vamos a la dirección :014F46B8 en la ventana de código vemos que no es así pues no hay rastros de esa api y si de otra más conocida, jeje:

```
Address   Hex dump     Disassembly                    Comment
014F46B8   55          push    ebp
014F46B9   8BEC        mov     ebp,esp
014F46BB   8B55 0C     mov     edx,[ebp+C]
014F46BE   8B45 08     mov     eax,[ebp+8]
014F46C1   3B05 F823500 cmp    eax,[15023F8]
014F46C7 v 75 09       jnz     short 014F46D2          api correcta :)
014F46C9   8B0495 F8235 mov    eax,[edx*4+15023F8]
014F46D0 v EB 07       jmp     short 014F46D9
014F46D2   52          push    edx
014F46D3   50          push    eax
014F46D4   E8 2710FFFF call    014E5700                jmp to kernel32.GetProcAddress
014F46D9   5D          pop     ebp
014F46DA   C2 0800     ret     8
014F46DD   8D40 00     lea     eax,[eax]
```

vaya¡!!! Una vieja conocida siempre dando problemas, grrr.. si quitamos el BPM que tenemos en la tabla de saltos (que es más grande a lo que abarca el BPM) y ponemos un BPM ON MEMORY - WRITE en toda la sección de código y vemos que aquí no vuelve a parar más, esta zona solamente se utiliza para esta api, donde sí para es en una última zona que es la siguiente:

```
Address   Hex dump     Disassembly                    Comment
0173055B   034D 04     add     ecx,[ebp+4]
0173055E   56          push    esi
0173055F   8F01        pop     dword ptr [ecx]        01519448
01730561 v EB 02       jmp     short 01730565
01730563   CD 20       int     20
01730565   68 6A1D4900 push    491D6A
```

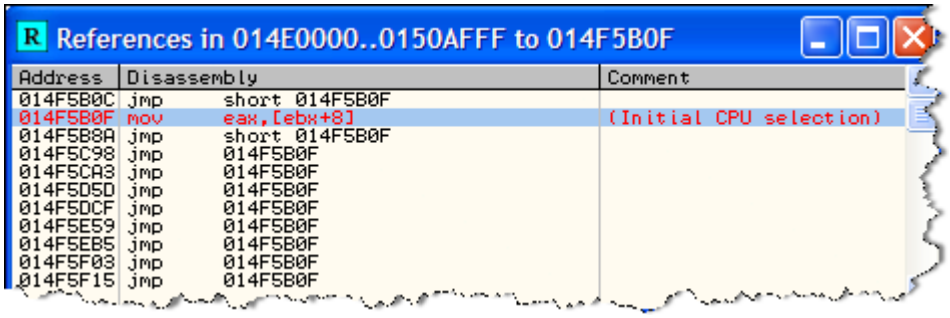
Si miramos los registros vemos:

```
Registers (MMX)
EAX 7F000000 GDI32.7F000000
ECX 00407C0E MetaRepo.00407C0E
EDX 0012FED8
EBX 7F000000 GDI32.7F000000
ESP 0012FEAC
EBP 0012FF28
ESI 01519448 <---- bytes del JMP
EDI 7F02238F GDI32.CloseEnhMetaFile
```

En este último caso tenemos los registros cambiados respecto a las anteriores zonas, aquí la api la encontramos en EDI y el valor que se salvará en [ECX] proviene de la pila y que contiene ESI, si nos fijamos una línea antes del POP hay un PUSH ESI que envía su contenido a la pila y que después sacará.

Por mucho que vayamos pulsando F9 no descubriremos más zonas, estas son todas las que utiliza, en realidad sí que podemos encontrar un par de ellas más pero no se utilizan en este programa.

Una cosa que antes comenté del salto hacia :014F5B0F donde podría estar el punto gatillo, esto lo comenté porque después de guardar en cada una de estas zonas encontramos un salto a dicha dirección, si vamos hasta allí con GO TO - EXPRESSION y hacemos FIND REFERENCES TO - SELECTED COMMAND veremos esto:



Esos son los saltos que nos envían hacia allí por eso comentaba que se podría utilizar como punto gatillo del script, es decir que se podría lanzar desde allí mismo, pero esta es otra historia que luego veremos cómo resolveremos.

De momento tenemos localizados los 5 puntos donde escribe en la tabla de saltos a la iat (JMP Table) y sabemos los valores que se manejan en cada uno de estos puntos por lo que podemos hacerle el script a mismo XDDD

Nuestros amigos los scripts ☺

Pues así es amigos, en este momento una buena opción pasa por un buen injerto o un buen script si no queremos estar toda una tarde escribiendo código, cosa nada recomendable si se puede evitar como parece ser el caso.

La cuestión es programar un script que detenga la ejecución en cada uno de los puntos encontrados y realice el trabajo adecuado a cada caso, pero tenemos el problema de que son 5 puntos y contamos únicamente con 4 HBPs, como sabemos que el punto :014f5ea2 era especial para el caso GetProcAddress como vimos antes así que el script no tendrá en cuenta esta entrada con lo que tendremos 4 puntos y 4 HBPs que ya nos gusta más, cuando el script termine repararemos a mano la entrada restante y que sabemos que será GetProcAddress.

Y dicho esto podemos comenzar, he de decir que he hecho algunas pruebas y he visto que todos los scripts que hacía me fallaban en dos saltos que son:

```
004013F8  - FF25 748C5101   jmp     [1518C74]
004079E4  - FF25 0C915101   jmp     [151910C]
```

Van ambos a GetProcAddress, el caso es que en el primero de :4013F8 hay veces que no falla y lo pasa bien pero en el segundo siempre falla ya que no sé porqué no repara los saltos siguientes a éste, es decir repara todos los saltos que hay desde el primero de ellos que está en :401328 hasta el primer salto a GetProcAddress de :4013F8 a partir de aquí había veces que fallaba el script y terminaba sin resolver el resto, otras veces pasaba bien la zona y reparaba también los saltos entre :4013F8 y :4079E4 donde está el segundo

salto a GetProcAddress, a partir de ahí no lograba que reparara el resto de saltos, la cuestión es que no he encontrado aún la causa pero es así.

Ante tal situación estuve tentado de realizar dos scripts, de hecho así lo hice el primero reparaba de :401328 hasta :4079E4 y el segundo desde :4079E4 hasta el último que está en :5E8CFC, después hacia un BINARY COPY y un BINARY PASTE del resto de saltos de un Olly a otro y así, pero.. no me convencía!!!

Entonces.. cuál es la solución? Supongo que habrá muchas yo encontré una, la clave está en un comando de OllyScript, jeje..

Desvelando el secreto..

Bueno es por darle un poco de intriga al tema, je.. como decía anteriormente el script no chuta (sin causa conocida) ante esta situación se me ocurrió detener su ejecución justo antes de la llegada a :4079E4 pero si detenía el script luego debía lanzarlo de nuevo por lo que no servía, para ello existe un comando PAUSE que nos da la posibilidad de pausar el script inclusive de modo condicional, esto es lo que haremos lo detendremos justo en la parte en que trata el salto que hay en la dirección :4079E4

Pongo el script a continuación:

```
var flag                    // será utilizada como bandera
var api                     // en ella guardaremos la api
var base                    // aquí salvamos la base de la DLL
var pos_iat                 // será un puntero a la iat
var aux                     // variable auxiliar_1
var aux1                    // variable auxiliar_2
var ayu                     // variable auxiliar_3
var otra                    // variable auxiliar_4
var direccion               // variable auxiliar_5

mov pos_iat, 63f000         // donde ubicaremos la tabla
mov flag, 0                 // se activará tras guardar la primera api

inicio:
dbh                         // ocultamos el debugger
bp 631BF8                   // ponemos un BP en el OEP para parar allí
bphws 014f3AEE, "x"         // colocamos el primer HBP
bphws 014f5E4F, "x"         // colocamos el segundo HBP
bphws 014f5D53, "x"         // colocamos el tercer HBP
bphws 01730561, "x"         // colocamos el cuarto HBP
eob vamos                   // nos preparamos para cambiar de etiqueta
run                         // dejamos correr el programa

vamos:
cmp eip, 014f3AEE           // estamos en :01503AEE?
je calls                    // vamos a la zona de calls
cmp eip, 014f5E4F           // estamos en :01505E4F?
je jmps                     // vamos a la zona de JMPs
cmp eip, 014f5D53           // estamos en :01505D53?
je jmps1                    // vamos igual a la zona de JMPs
cmp eip, 01730561           // estamos en :0173055F?
je push                     // vamos a la zona de PUSHes
jmp termina                 // si no estamos en ninguno terminamos
```

```
// Procedimiento CALLs
calls:
mov api, ebp                // obtenemos la api que está en EBP
log api                     // la logueamos
cmp flag, 0                 // es la primera vez?
jne otra_vez                // guardamos directamente en la tabla
GMI api, MODULEBASE         // Obtenemos su imagebase
mov base, $RESULT           // y la guardamos en "base"

iguales:
mov aux, ebx                // dirección del salto a la variable aux
mov direccion, ebx          // obtenemos la dirección del salto
cmp direccion, 4079dc       // estamos en el que nos interesa?
je pausamos                 // si es así pausamos el script
log direccion               // logueamos el valor por si acaso
mov [pos_iat], api          // salvamos la api en la iat
mov [aux], #FF25#           // reparamos el call
add aux, 2                  // reajustamos el puntero
mov [aux], pos_iat          // y lo dejamos apuntando a la iat
log pos_iat                 // logueamos su valor
add pos_iat, 4              // apuntamos al siguiente DWORD
mov flag, 1                 // activamos la bandera
mov eip, 014F3AF4           // salteamos el EIP
jmp inicio                  // vamos a la etiqueta inicio

otra_vez:
GMI api, MODULEBASE         // Obtenemos su imagebase
cmp base, $RESULT           // y la comparamos con "base"
je iguales                  // si son del mismo tipo la guardamos
mov base, $RESULT           // sino actualizamos la variable
add pos_iat, 4              // dejamos separación de ceros 00000000
jmp iguales                 // y vamos a salvarla igualmente


// Procedimiento jmps
jmps:
mov direccion, esi          // obtenemos la dirección del salto+2
sub direccion, 2            // obtenemos la dirección del salto
cmp direccion, 4079dc       // estamos en el que nos interesa?
je pausamos                 // si es así pausamos el script
log direccion               // logueamos el valor por si acaso
mov api, ebp                // guardamos la api en su variable
log api                     // y la logueamos
cmp flag, 0                 // miramos si es la primera pasada
jne sgute                   // saltará si no es la primera
GMI api, MODULEBASE         // Obtenemos su imagebase
mov base, $RESULT           // y la guardamos en "base"

salvo:
mov otra, esi               // obtenemos la dirección del salto+2
mov [pos_iat], api          // guardamos la api en la iat
mov [otra], pos_iat         // hago que el JMP apunte a su sitio
log pos_iat                 // logueamos el destino del salto
add pos_iat, 4              // apuntamos al siguiente DWORD
mov flag, 1                 // activamos la bandera
mov eip, 014F5E51           // modificamos el EIP
jmp inicio                  // vamos a la etiqueta inicio
```

```
sgute:
GMI api, MODULEBASE          // Obtenemos su imagebase
cmp base, $RESULT            // y la comparamos con "base"
je salvo                     // si son del mismo tipo salvamos
mov base, $RESULT            // sino actualizamos la variable
add pos_iat, 4               // y dejamos una separación de ceros
jmp salvo                    // y vamos a salvarla


// Procedimiento jmps1
jmps1:
mov direccion, esi           // obtenemos la dirección del salto+2
sub direccion, 2             // obtenemos la dirección del salto
cmp direccion, 4079dc        // estamos en el que nos interesa?
je pausamos                  // si es así pausamos el script
log direccion                // logueamos el valor por si acaso
mov api, ebp                 // guardamos la api en su variable
log api                      // y la logueamos
cmp flag, 0                  // miramos si es la primera pasada
jne sgute1                   // saltará si no es la primera
GMI api, MODULEBASE          // Obtenemos su imagebase
mov base, $RESULT            // y la guardamos en "base"

salvo1:
mov aux1, esi                // obtenemos la dirección del salto+2
mov [pos_iat], api           // guardamos la api en la iat
mov [aux1], pos_iat          // hago que el JMP apunte a su sitio
log pos_iat                  // logueamos valor de destino del salto
add pos_iat, 4               // apuntamos al siguiente DWORD
mov flag, 1                  // activamos la bandera
mov eip, 014f5D55            // modificamos el EIP
jmp inicio                   // vamos a la etiqueta inicio

sgute1:
GMI api, MODULEBASE          // Obtenemos su imagebase
cmp base, $RESULT            // y la comparamos con "base"
je salvo1                    // si son del mismo tipo salvamos
mov base, $RESULT            // sino actualizamos la variable
add pos_iat, 4               // y dejamos una separación de ceros
jmp salvo1                   // vamos a la etiqueta salvo1

// Procedimiento PUSHes
push:
mov direccion, ecx           // obtenemos la dirección del salto+2
sub direccion, 2             // obtenemos la dirección del salto
cmp direccion, 4079dc        // estamos en el que nos interesa?
je pausamos                  // si es así pausamos el script
log direccion                // logueamos el valor por si acaso
mov api, edi                 // guardamos la api en su variable
log api                      // y la logueamos
cmp flag, 0                  // miramos si es la primera pasada
jne sigo                     // saltará si no es la primera
GMI api, MODULEBASE          // Obtenemos su imagebase
mov base, $RESULT            // y la guardamos en "base"

salvar:
mov ayu, ecx
mov [pos_iat], api           // guardamos la api en la iat
mov [ayu], pos_iat           // hago que el JMP apunte a su sitio
```

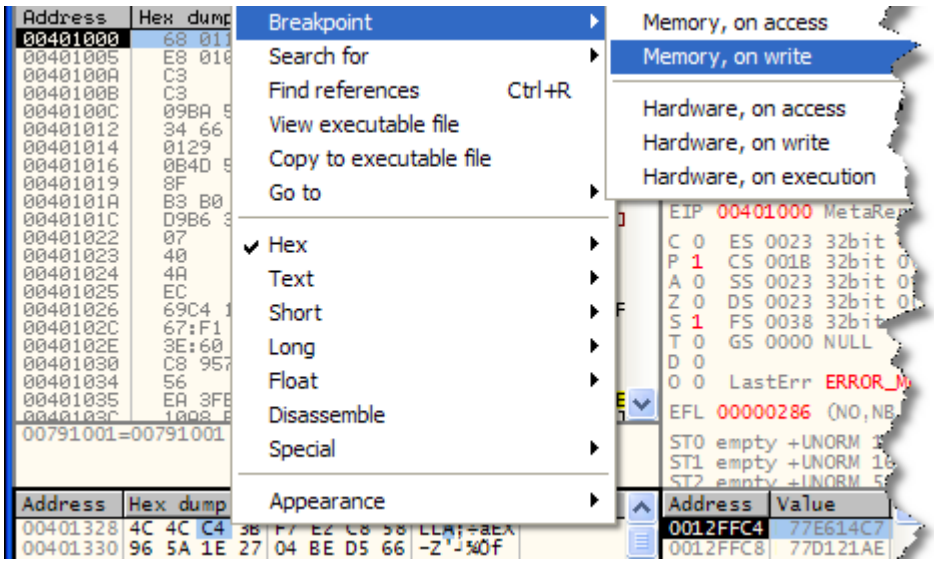
```
log pos_iat                 // logueamos valor de destino del salto
add pos_iat, 4              // apuntamos al siguiente DWORD
mov flag, 1                 // activamos la bandera
jmp inicio                  // vamos a la etiqueta inicio

sigo:
GMI api, MODULEBASE         // Obtenemos su imagebase
cmp base, $RESULT           // y la comparamos con "base"
je salvar                   // si son del mismo tipo salvamos
mov base, $RESULT           // sino actualizamos la variable
add pos_iat, 4              // y dejamos una separación de ceros
jmp salvar                  // vamos a la etiqueta salvar

pausamos:
pause                       // lo dejamos en pausa para que funcione
add pos_iat, 4              // reajustamos el puntero de tabla
jmp inicio                  // vamos a la etiqueta inicio

termina:
bc 631BF8
bphwc 014f3AEE              // quitamos el primer HBP
bphwc 014f5E4F              // quitamos el segundo HBP
bphwc 014f5D53              // quitamosel tercer HBP
bphwc 01730561              // quitamos el cuarto HBP
MSG "Dumpea ahora y repara la iat con el ImpRec :)"
ret                         // retornamos el control a Olly
```

Puede parecer más complicado de lo que realmente es, pero es de lo más simple y no lo vamos a explicar porque ya está hecho en los comentarios agregados que lo desglosan línea por línea, a grandes rasgos se puede decir que intercepta el momento en que el packer rompe cada uno de los saltos (0xFF25h) e introduce la api tratada en la iat que ubicaremos a partir de :69f000 donde hay sitio de sobra y dejará el JMP apuntando a la dirección donde salvó la citada api. El punto gatillo del script lo encontramos así de fácil: Reiniciamos Olly con CTRL+F2 hacemos GOTO - EXPRESSION y vamos a la dirección :401328 en el DUMP:



Ahí está el primer JMP al que le colocamos un BPM ON MEMORY - WRITE a partir del tercer byte para esquivar el opcode de salto 0xFF25h que no nos interesa perseguir, vamos pulsando F9 y Olly hace un par de paradas en dos lugares poco interesantes, a la tercera se detiene donde queremos:

```
Address   Hex dump      Disassembly                          Comment
014F5E37  75 0C         jnz     short 014F5E45
014F5E39  837B 34 00    cmp     dword ptr [ebx+34],0
014F5E3D  74 06         je      short 014F5E45
014F5E3F  8B6B 34       mov     ebp,[ebx+34]
014F5E42  036B 40       add     ebp,[ebx+40]
014F5E45  8B4424 0C     mov     eax,[esp+C]
014F5E49  8928          mov     [eax],ebp
014F5E4B  8B4424 0C     mov     eax,[esp+C]                  // situamos aqui el EIP
014F5E4F  8906          mov     [esi],eax                    // aqui paro
014F5E51  0FB74424 04   movzx   eax,word ptr [esp+4]
014F5E56  0143 08       add     [ebx+8],eax
```

El Olly se detiene en :014f5e4f donde va a escribir los bytes del primer salto que están en EAX, aprovechamos que la tabla de saltos está intacta en este momento para utilizar este punto como gatillo del script, pero antes para que funcione debemos situar el EIP una instrucción antes con un NEW ORIGIN HERE en dicha instrucción, y lanzar el script PLUGINS – OLLYSCRIPT – RUN SCRIPT…

Al poco tiempo se detiene aquí:

```
Address   Hex dump      Disassembly                          Comment
014F5D53  8906          mov     [esi],eax
014F5D55  0FB74424 04   movzx   eax,word ptr [esp+4]
014F5D5A  0143 08       add     [ebx+8],eax
014F5D5D  E9 ADFDFFFF   jmp     014F5B0F
014F5D62  3C 04         cmp     al,4
014F5D64  0F85 F4000000 jnz     014F5E5E
014F5D6A  EB 01         jmp     short 014F5D6D
014F5D6C  E9 8B43088A   jmp     8B57A0FC
014F5D71  00FF          add     bh,bh
014F5D73  43            inc     ebx
014F5D74  0884C0 755B8B43 or    [eax+eax*8+438B5B75],al
```

Veamos dónde dejó el trabajo:

```
Address   Hex dump      Disassembly                          Comment
004079C2  8BC0          mov     eax,eax
004079C4  FF25 40F26300 jmp     [63F240]                     kernel32.GetModuleFileNameA
004079CA  8BC0          mov     eax,eax
004079CC  FF25 44F26300 jmp     [63F244]                     kernel32.GetModuleHandleA
004079D2  8BC0          mov     eax,eax
004079D4  FF25 48F26300 jmp     [63F248]                     kernel32.GetModuleHandleA
004079DA  8BC0          mov     eax,eax
004079DC  FF25 701EB310 jmp     [10B31E70]
004079E2  8BC0          mov     eax,eax
004079E4  FF25 35DCB92A jmp     [2AB9DC35]
004079EA  8BC0          mov     eax,eax
004079EC  0000          add     [eax],al
004079EE  0E            push    cs
004079EF  25 DD3C8BC0   and     eax,C08B3CDD
```

Justo donde queríamos, repararemos ese salto a mano y dejaremos correr el script de nuevo. Si miramos en la ventana de registros vemos esto:

```
Registers (FPU)
EAX 01519100
ECX 00000000        api correcta :)
EDX 01519100
EBX 015103E0
ESP 0012FEF8
EBP 77E52897 kernel32.GetPrivateProfileStringA
ESI 004079DE MetaRepo.004079DE
```

Salvamos esta api en la dirección siguiente de la iat:

```
Address   Hex dump                                         ASCII
0063F20C  06 DD E4 77 06 DD E4 77 43 91 E4 77 3F 97 E4 77  -Ÿäw-ŸäwC'äw?-
0063F21C  1F 95 E5 77 C1 27 E5 77 34 22 E6 77 00 00 00 00  •åwÁ'åw4"æw....
0063F22C  02 15 F4 77 00 00 00 00 68 5A E5 77 62 73 E5 77  ¬-ôw....hZåwbs
0063F23C  67 DA E4 77 A9 AD E5 77 86 AD E5 77 86 AD E5 77  gÚäw©-åw†-åw†-
0063F24C  97 28 E5 77 00 00 00 00 00 00 00 00 00 00 00 00  -(åw..........
0063F25C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0063F26C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0063F27C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

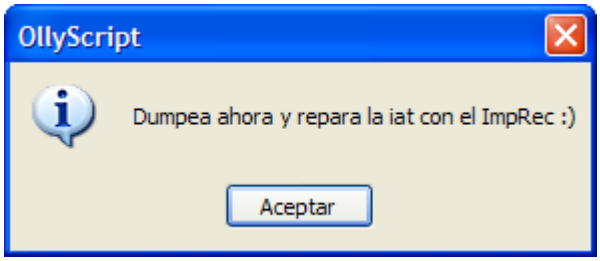
Y dejamos el salto apuntando a dicha entrada:

```
Address   Hex dump        Disassembly          Comment
004079D4  - FF25 48F26300 jmp   [63F248]       kernel32.GetModuleHandleA
004079DA    8BC0          mov   eax,eax
004079DC  - FF25 4CF26300 jmp   [63F24C]       kernel32.GetPrivateProfileStringA
004079E2    8BC0          mov   eax,eax
004079E4    FF25 35DCB92A jmp   [2AB9DC35]
004079EA    8BC0          mov   eax,eax
004079EC    0000          add   [eax],al
004079EE    0E            push  cs
004079EF    25 DD3C8BC0   and   eax,C08B3CDD
```

Y ya podemos continuar, para que no se eche a perder el trabajo debemos saltear el EIP hacia la siguiente instrucción :014f5d55 para que no chafe el salto reparado manualmente y poner un BPM ON MEMORY - WRITE en el salto siguiente en :4079E4, pulsamos F9 y para en la zona especial donde lo escribe:

```
Address   Hex dump     Disassembly          Comment
014F5EA2    8906       mov   [esi],eax      // caso especial GetProcAddress
014F5EA4    8A4C24 06  mov   cl,[esp+6]
014F5EA8    8B5424 14  mov   edx,[esp+14]
014F5EAC    8B4424 0C  mov   eax,[esp+C]
014F5EB0    E8 FFF0FFFF call  014F4FB4
014F5EB5  ^ E9 55FCFFFF jmp   014F5B0F
014F5EBA    8BC3       mov   eax,ebx
014F5EBC    E8 73EDFFFF call  014F4C34
```

Ahora la cosa es tan sencilla como dejar correr el script nuevamente pulsando en PLUGINS - OLLYSCRIPT - RESUME y esperar que pare en el OEP, irá más despacio a causa del cambio realizado que ralentiza todo, cuando termina vemos:



Y otro que dice "Script finished", tenemos los dos saltos a GetProcAddress por reparar, insertamos la api al final de la tabla e introducimos la api:

```
Address   Hex dump                                          ASCII
0063F954  66 B0 88 00 4A B0 88 00 4F 49 85 00 43 3F 81 00  f°^.J°^.OI….C?
0063F964  00 00 00 00 90 23 36 76 CF 90 36 76 05 31 37 76  ....#6vI6v|17
0063F974  0F DA 37 76 BF 6F 36 76 00 00 00 00 A8 67 E9 77  ÚÚ7v¿o6v.... gé
0063F984  50 E4 E8 77 B6 63 E9 77 A1 39 E5 77 00 00 00 00  Päèw¶céw¡9åw..
0063F994  A4 11 74 76 00 00 00 00 32 B3 E5 77 00 00 00 00  ¤◄tv....2³åw..
0063F9A4  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

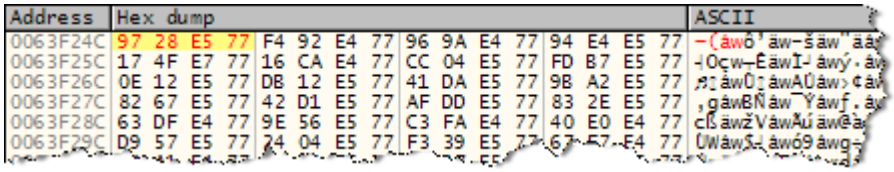
Y dejamos los dos saltos apuntando a ella:

```
Address   Hex dump        Disassembly          Comment
004013EE    8BC0          mov   eax,eax
004013F0  - FF25 84F06300 jmp   [63F084]       kernel32.GetModuleHandleA
004013F6    8BC0          mov   eax,eax
004013F8  - FF25 9CF96300 jmp   [63F99C]       kernel32.GetProcAddress
004013FE    8BC0          mov   eax,eax
00401400  - FF25 88F06300 jmp   [63F088]       kernel32.GetStartupInfoA
00401406    8BC0          mov   eax,eax
00401408  - FF25 8CF06300 jmp   [63F08C]       kernel32.GetThreadLocale
```

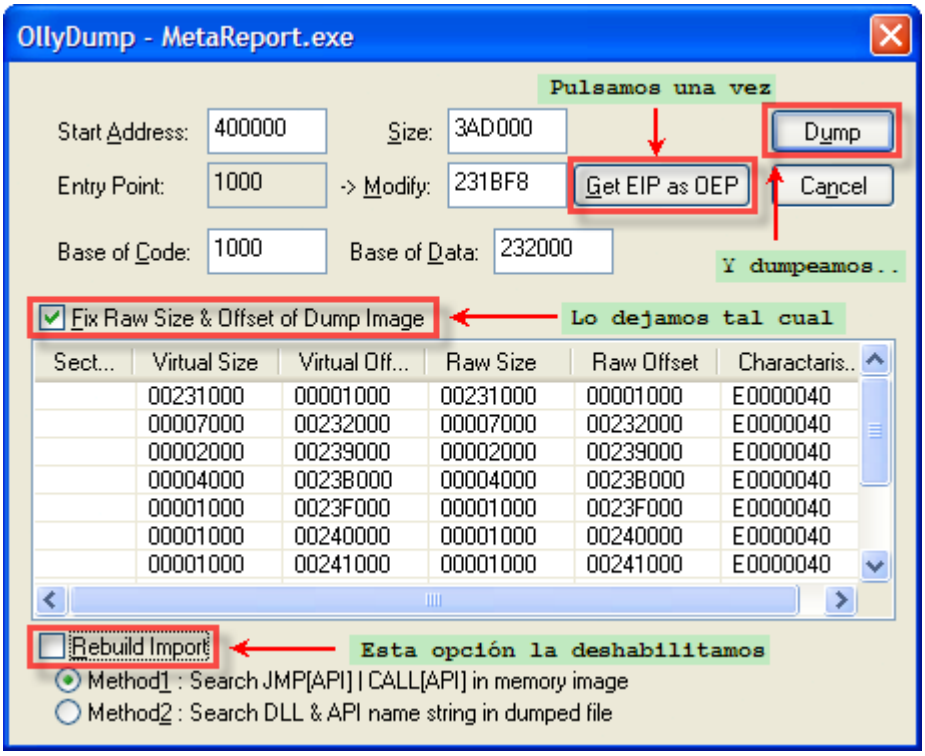
Ahora el segundo de ellos:

```
Address   Hex dump        Disassembly          Comment
004079D4  - FF25 48F26300 jmp   [63F248]       kernel32.GetModuleHandleA
004079DA    8BC0          mov   eax,eax
004079DC  - FF25 4CF26300 jmp   [63F24C]       kernel32.GetPrivateProfileStringA
004079E2    8BC0          mov   eax,eax
004079E4  - FF25 9CF96300 jmp   [63F99C]       kernel32.GetProcAddress
004079EA    8BC0          mov   eax,eax
004079EC  - FF25 50F26300 jmp   [63F250]       kernel32.GetProfileStringA
004079F2    8BC0          mov   eax,eax
```

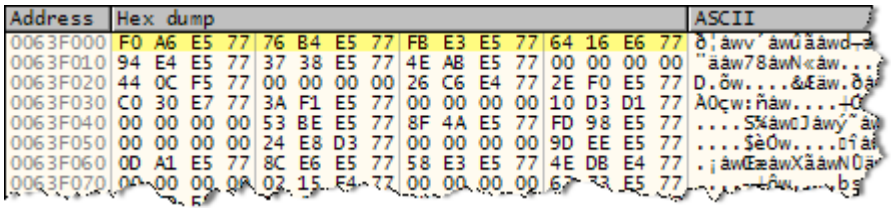
Justo encima vemos el JMP hacia GetPrivateProfileStringA que reparamos a mano cuando el script se pausó para no fallar, además pondremos la api en la iat:



Ahora dumpeamos pulsando en PLUGINS - OLLYDUMP - DUMP DEBUGGED PROCESS en el cuadro que aparece elegimos las siguientes opciones:



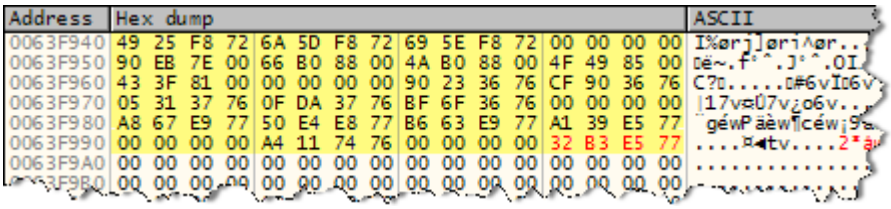
Al dumpeado lo podemos llamar dumpeado.exe o como queramos eso no importa, ahora abrimos el ImpRec y seleccionamos el proceso en ejecución desde la pestaña ATTACH TO AN ACTIVE PROCES, el que se llama metareport.exe es el nuestro, lo cargará sin mayores problemas. El siguiente paso será delimitar la iat que hemos colocado a partir de :63f000, veámosla:



Ese es el inicio lógicamente, lo anotamos:

inicio_iat: 63f000

Veamos el final:



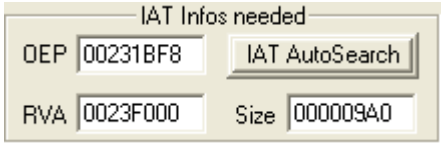
Lo anotamos también por su puesto:

final_iat: 63f9a0

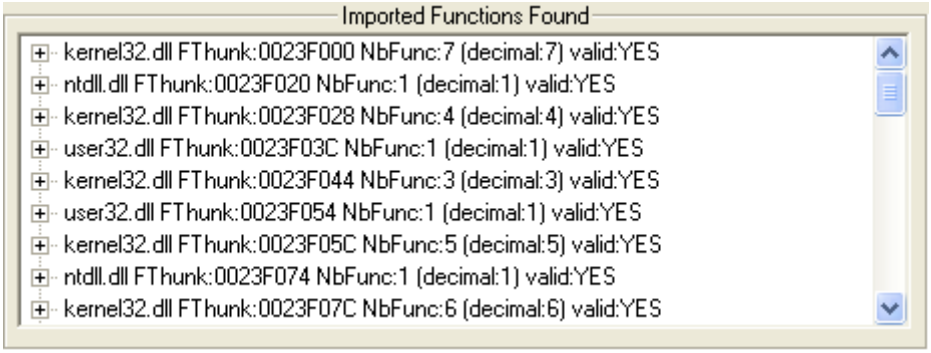
Conociendo estos valores podemos obtener fácilmente su largo:

largo: 9a0

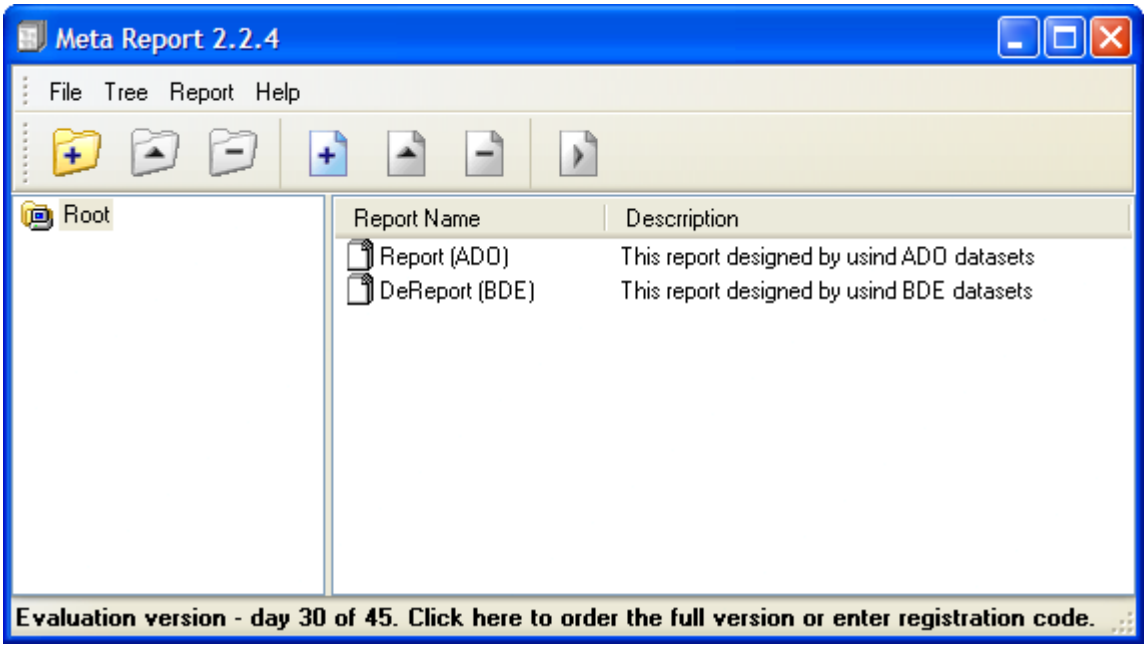
Estos valores son fundamentales para que el ImpREC funcione, siempre tenemos la opción de buscarlos automáticamente con IAT AUTOSEARCH pero así estamos más seguros, los colocamos:



Picamos en GET IMPORTS y nos saca todas las funciones XDD



Las tenemos TODAS, jeje.. pulsamos en FIX DUMP y elegimos dumpeado.exe o como lo hayamos llamado, nos dejará dumpeado_.exe lo ejecutamos y..



funciona perfectamente, podemos utilizar todas las funciones y el programa responde bien así que es de suponer que está bien desempakado ☺

Y ya que estamos vamos a ver si podemos crackearlo..

Terminando lo empezado..

Pues eso mismo es lo que vamos a intentar de hacer, se podría hacer una segunda parte del tute para las limitaciones pero mejor un pequeño anexo aquí mismo, el desempakado está finalizado así que lo que veamos de aquí en adelante será propio del programa y no del packer.

He de decir que no sé muy bien cómo funciona este programa pues no consigo crear un reporte en condiciones, hay dos creados con el tipo ADO y BDE respectivamente pero que todavía no sé cómo se hacen ;(

En cualquier caso vamos a tirar para adelante con esto a ver qué sale, mirando entre las STRING REFERENCES vemos cositas interesantes como estas:

```
Address   Hex dump   Disassembly                          Comment
0062BF48  .  8338 00   cmp     dword ptr [eax],0
0062BF4B  .v 75 25     jnz     short 0062BF72
0062BF4D  .  8D4D EC   lea     ecx,[ebp-14]
0062BF50  .  BA A4640000 mov   edx,64A4
0062BF55  .  B8 58C16200 mov   eax,0062C158          ASCII "434465071455824555575 42
0062BF5A  .  E8 FD67FBFF call  005E275C
0062BF5F  .  8B55 EC   mov     edx,[ebp-14]
0062BF62  .  8B83 0803000( mov eax,[ebx+308]
0062BF68  .  E8 DBFAE5FF call  0048BA48
0062BF6D  .v E9 28010000 jmp   0062C09A
0062BF72  >  8D45 E8   lea     eax,[ebp-18]
0062BF75  .  50        push    eax
0062BF76  .  8D4D E4   lea     ecx,[ebp-1C]
0062BF79  .  BA A4640000 mov   edx,64A4
0062BF7E  .  B8 94C16200 mov   eax,0062C194          ASCII "452345121354025517353400
0062BF83  .  E8 D467FBFF call  005E275C
0062BF88  .  8B45 E4   mov     eax,[ebp-1C]
0062BF8B  .  50        push    eax
0062BF8C  .  A1 9C876300 mov   eax,[63879C]
0062BF91  .  8B00      mov     eax,[eax]
0062BF93  .  8945 F0   mov     [ebp-10],eax
0062BF96  .  C645 F4 0B mov    byte ptr [ebp-C],0B
0062BF9A  .  8D55 F0   lea     edx,[ebp-10]
0062BF9D  .  33C9      xor     ecx,ecx
0062BF9F  .  58        pop     eax
0062BFA0  .  E8 77F4DDFF call  0040B41C              dumpeado.0040B41C
0062BFA5  .  8B55 E8   mov     edx,[ebp-18]
0062BFA8  .  8B83 0803000( mov eax,[ebx+308]
```

Un poco más debajo de estas cadenas encontramos esto:

```
Address   Hex dump   Disassembly                          Comment
0062BFDD  .  E8 36FAE5FF call  0048BA18
0062BFE2  .  8B45 CC   mov     eax,[ebp-34]
0062BFE5  .  8945 D0   mov     [ebp-30],eax
0062BFE8  .  C645 D4 0B mov    byte ptr [ebp-2C],0B
0062BFEC  .  B8 C8C16200 mov   eax,0062C1C8          ASCII "Site License"
0062BFF1  .  8945 D8   mov     [ebp-28],eax
0062BFF4  .  C645 DC 0B mov    byte ptr [ebp-24],0B
0062BFF8  .  8D55 D0   lea     edx,[ebp-30]
0062BFFB  .  B9 01000000 mov   ecx,1
0062C000  .  B8 E0C16200 mov   eax,0062C1E0          ASCII "%s\n%s"
0062C005  .  E8 12F4DDFF call  0040B41C              dumpeado.0040B41C
0062C00A  .  8B55 E0   mov     edx,[ebp-20]
0062C00D  .  8BC6      mov     eax,esi
0062C00F  .  E8 34FAE5FF call  0048BA48
```

Parece que en esta rutina se hace algo importante, hay varias cadenas más que podrían ser interesantes, también tenemos la alternativa de introducir un serial desde HELP - ENTER REGISTRATION CODE si introducimos uno cualquiera nos dice que reiniciemos el programa, previsiblemente se valide al inicio y por eso pide reiniciar, también podría estar jugando al despiste y validarlo ahora, mirando por el código también vemos cositas como esta:

```
00621797  |.  B8 9C186200   |mov     eax,0062189C        ; ASCII "KEY BEGIN KEY"
0062179C  |.  E8 FF3CDEFF   |call    004054A0
006217A1  |.  85C0          |test    eax,eax
006217A3  |.  74 58         |je      short 006217FD
```

y esta:

```
006217CE  |.  B8 B4186200   ||mov     eax,006218B4        ; ASCII "KEY END KEY"
006217D3  |.  E8 C83CDEFF   ||call    004054A0
006217D8  |.  85C0          ||test    eax,eax
006217DA  |.  75 29         ||jnz     short 00621805
```

Esto tiene pinta de rutina verificadora del serial, jeje.. hay otra más:

```
00621986   .  B8 F8196200   mov     eax,006219F8          ; ASCII "KEY BEGIN KEY"
0062198B   .  E8 103BDEFF   call    004054A0
00621990   .  85C0          test    eax,eax
00621992   .  75 1E         jnz     short 006219B2
```

A ver si hallamos la forma de romper la lata, tenemos que al pulsar sobre el botón ABOUT Olly nos detiene aquí:

```
Address   Hex dump   Disassembly                    Comment
0062BEC8  .  55      push    ebp                    // rutrina verificacion?
0062BEC9  .  8BEC    mov     ebp,esp
0062BECB  .  B9 09000000  mov  ecx,9
0062BED0  >  6A 00   push    0
0062BED2  .  6A 00   push    0
0062BED4  .  49      dec     ecx
0062BED5  .^ 75 F9   jnz     short 0062BED0
0062BED7  .  51      push    ecx
0062BED8  .  53      push    ebx
```

Más para abajo están las cadenas esas tan largas "43446507145582455557.." y el "Site Lincese" el "x licenses" el "Single License" etc., no es muy complicado darse cuenta que aquí decide el tipo de licencia que tenemos si es que la tenemos, jeje.. así que veremos qué busca y si se lo podemos dar ;)

Una de las cosas que más me llamó la atención fue:

```
Address   Hex dump   Disassembly                    Comment
0062BF3E  .  E8 05FBE5FF  call  0048BA48
0062BF43  .  A1 9C876300  mov   eax,[63879C]
0062BF48  .  8338 00 cmp   dword ptr [eax],0        // lee de [637eec]
0062BF4B  .v 75 25   jnz     short 0062BF72
0062BF4D  .  8D4D EC lea     ecx,[ebp-14]
0062BF50  .  BA A4640000  mov   edx,64A4
0062BF55  .  B8 58C16200  mov   eax,0062C158         ASCII "434465071455824555575 4
0062BF5A  .  E8 FD67FBFF  call  005E275C
0062BF5F  .  8B55 EC mov     edx,[ebp-14]
0062BF62  .  8B83 0803000 mov   eax,[ebx+308]
0062BF68  .  E8 DBFAE5FF  call  0048B948
```

Donde lee la variable [637eec] donde no hay nada:

```
Address   Hex dump                                          ASCII
00637EEC  00 00 00 00 00 00 00 00 00 00 00 00 C8 BA 62 00  ............È°b.
00637EFC  4C 16 5F DC E9 90 B3 41 B9 8A 3C 98 A2 93 5A E0  L__Üé*A'S<˜¢"Zà
00637F0C  3C 75 44 00 5C 75 44 00 C4 B1 50 00 AC B1 50 00  <uD.\uD.Ä±P.¬±P.
00637F1C  9C 8A 40 00 84 89 40 00 0C A7 63 00 0C 8A 55 00  œŠ@.„‰@..§c..ŠU.
00637F2C  4C AB 4F 00 7C B2 50 00 34 AF 50 00 E8 A8 41 00  L«O.|²P.4¯P.è¨A.
```

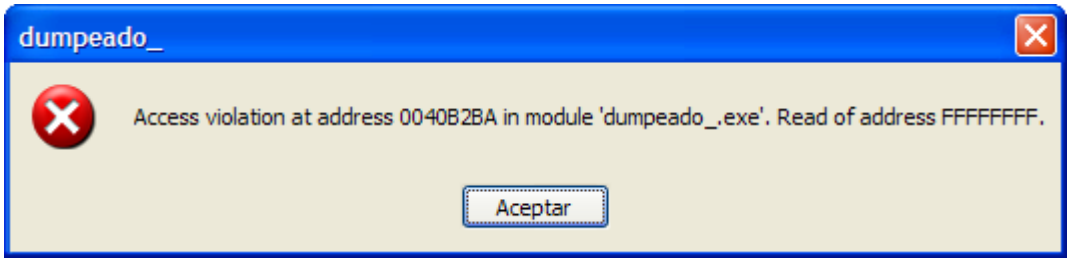
Si no forzamos el salto cambiando el valor de la variable [637eec] dejándola por ejemplo a 00000001h saltamos a esta otra zona:

```
0062BF7E   .  B8 94C16200   mov     eax,0062C194          ;/ ASCII "45234512135402551…"
0062BF83   .  E8 D467FBFF   call    005E275C
0062BF88   .  8B45 E4       mov     eax,[ebp-1C]          ;|
0062BF8B   .  50            push    eax                   ;|
0062BF8C   .  A1 9C876300   mov     eax,[63879C]          ;|
0062BF91   .  8B00          mov     eax,[eax]             ;|
0062BF93   .  8945 F0       mov     [ebp-10],eax          ;|
0062BF96   .  C645 F4 0B    mov     byte ptr [ebp-C],0B   ;|
0062BF9A   .  8D55 F0       lea     edx,[ebp-10]          ;|
0062BF9D   .  33C9          xor     ecx,ecx               ;|
0062BF9F   .  58            pop     eax                   ;|
0062BFA0   .  E8 77F4DDFF   call    0040B41C              ;\dumpeado.0040B41C
```

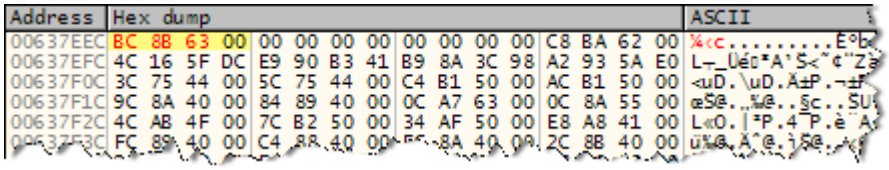
Si miramos los registros vemos:

```
Registers (FPU)
EAX 014C4D34 ASCII "Licensed to %s"
ECX 00000000
EDX 0012F950
EBX 014CCE44
ESP 0012F8FC
EBP 0012F960
```

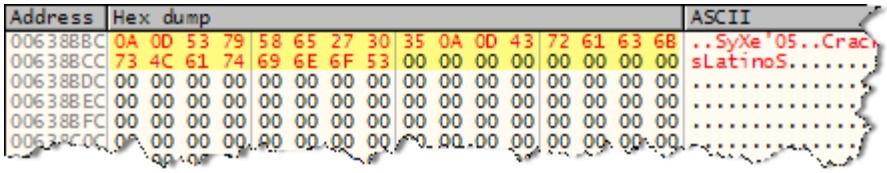
Vaya!!! Parece que vamos por el buen camino, pero si seguimos traceando llega un punto en que se genera un ACCESS_VIOLATION mostrando este mensaje:



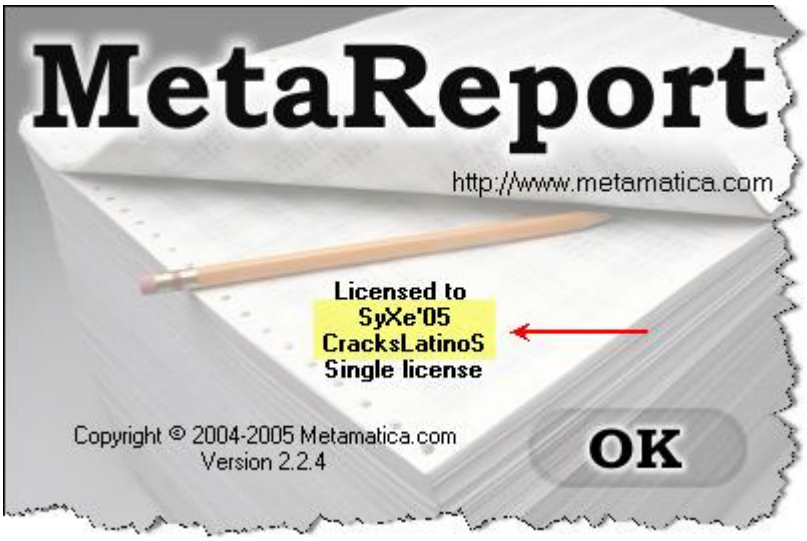
Si colocamos un BP en esta dirección veremos que intenta leer de una dirección de memoria inaccesible, en concreto [FFFFFFFD] y esto es a causa del valor que le hemos dado a la variable [637eec] pues resulta que la utiliza como puntero y claro se genera el error, se me ocurrió poner allí una dirección válida:



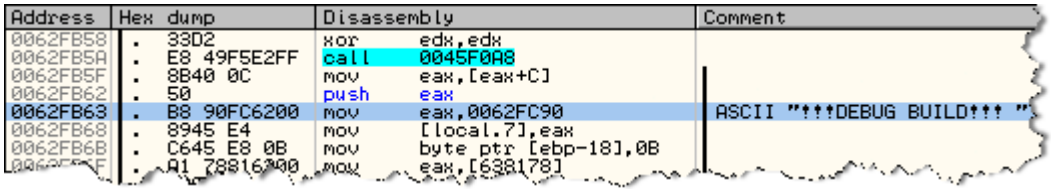
Y allí colocamos algunos datos:



El 0x0D0Ah es un salto de línea, ahora veremos porqué, colocando estos datos allí y dejando la variable [637eec] apuntando allí veremos esto en el ABOUT:



Nos reconoce como usuarios registrados, jeje.. salvemos los cambios al disco con COPY TO EXECUTABLE - ALL MODIFICATIONS y SAVE FILE y está muy bien porque vemos nuestro nombre en el About pero seguimos sin estar registrados, algo se nos está escapando, retornemos al código del programa, buscamos entre las STRING REFERENCES y vemos una que dice DEBUG VERSION, si hacemos doble clic nos lleva una de las dos zona donde ésta aparece:



Si miramos por encima vemos que valora una variable:

```
0062FB3E  |> \A1 CC816300   mov     eax,[6381CC]
0062FB43  |.  8038 00       cmp     byte ptr [eax],0
0062FB46  |.  74 79         je      short 0062FBC1
```

Aunque lo realmente interesante está más arriba:

```
Address   Hex dump    Disassembly                          Comment
0062FAE1  .  64:FF30  push    dword ptr fs:[eax]
0062FAE4  .  64:8920  mov     fs:[eax],esp
0062FAE7  .  A1 10806300  mov     eax,[638010]
0062FAEC  .  8338 00  cmp     dword ptr [eax],0            // ojo a este flag
0062FAEF  .v 74 4D    je      short 0062FB3E
0062FAF1  .  8B83 1C03000( mov    eax,[ebx+31C]
0062FAF7  .  8B80 0802000( mov    eax,[eax+208]
0062FAFD  .  33D2     xor     edx,edx
0062FAFF  .  E8 4415DFFF  call    00421048
0062FB04  .  8B83 1C03000( mov    eax,[ebx+31C]
0062FB0A  .  33D2     xor     edx,edx
0062FB0C  .  8990 2001000( mov    [eax+120],edx
```

Veamos dónde está leyendo, colocamos un BP en :62faec reiniciamos Olly con CTRL+F2 y cuando pare hacemos un FOLLOW IN DUMP - MEMORY ADDRESS y vemos esto:

```
Address   Hex dump                                         ASCII
00637EF4  00 00 00 00 C8 BA 62 00 4C 16 5F DC E9 90 B3 41  ....È°b.L-_Üé*A
00637F04  B9 8A 3C 98 A2 93 5A E0 3C 75 44 00 5C 75 44 00  'S<˜¢"Zà<uD.\uD
00637F14  C4 B1 50 00 AC B1 50 00 9C 8A 40 00 84 89 40 00  Ä±P.¬±P.œS@.„‰@.
00637F24  0C A7 63 00 0C 8A 55 00 4C AB 4F 00 7C B2 50 00  .§c..SU.L«O.|²P
00637F34  34 AF 50 00 E8 A8 41 00 FC 89 40 00 C4 88 40 00  4¯P.è¨A.ü‰@.Ä^@
00637F44  EC 8A 40 00 2C 8B 40 00 94 8B 40 00 E0 D2 42 00  ì<@.,‹@.”‹@.àÒB.
```

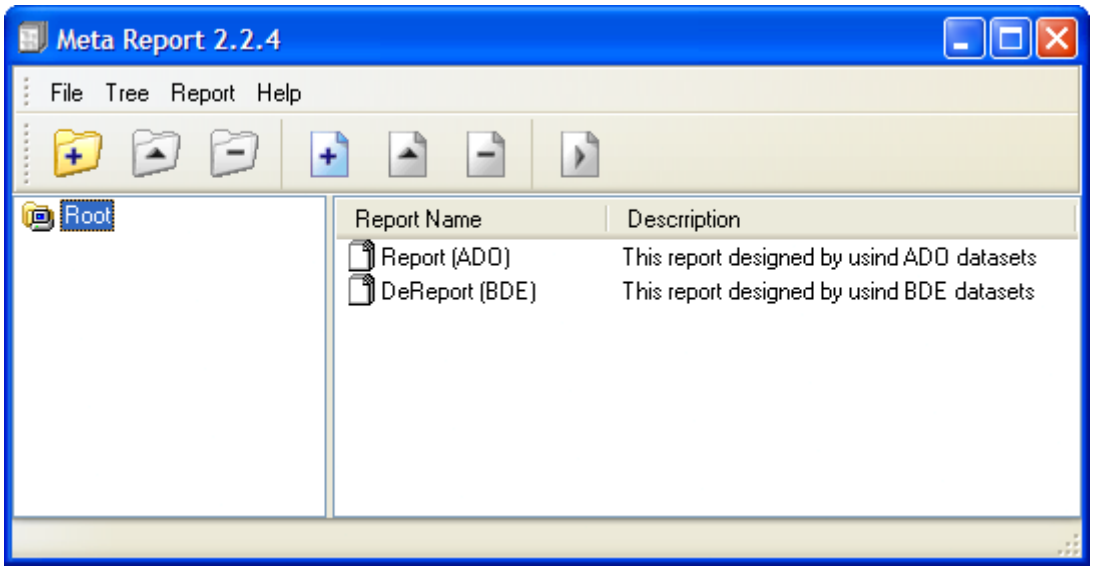
Si buscamos referencias a esta variable sólo encontramos una dónde ésta se modifique, es aquí:

```
0062B987  .  E8 50F3DDFF   call    0040ACDC
0062B98C  .  85C0          test    eax,eax
0062B98E  .  0F86 E9000000 jbe     0062BA7D
0062B994  .  C705 F47E6300>mov     dword ptr [637EF4],-1
0062B99E  .  B2 01         mov     dl,1
```

Haremos una cosa, reiniciamos nuevamente Olly y colocamos ese valor en ella:

```
Address   Hex dump                                         ASCII
00637EF4  FF FF FF FF C8 BA 62 00 4C 16 5F DC E9 90 B3 41  ÿÿÿÿÈ°b.L-_Üé*A
00637F04  B9 8A 3C 98 A2 93 5A E0 3C 75 44 00 5C 75 44 00  'S<˜¢"Zà<uD.\uD
00637F14  C4 B1 50 00 AC B1 50 00 9C 8A 40 00 84 89 40 00  Ä±P.¬±P.œS@.„‰@.
00637F24  0C A7 63 00 0C 8A 55 00 4C AB 4F 00 7C B2 50 00  .§c..SU.L«O.|²P
00637F34  34 AF 50 00 E8 A8 41 00 FC 89 40 00 C4 88 40 00  4¯P.è¨A.ü‰@.Ä^@
00637E44  EC 8A 40 00 2C 8B 40 00 94 8B 40 00 E0 D2 42 00  ì<@.,‹@.”‹@.àÒB.
```

Si pulsamos F9 arranca el programa sin mostrar cadenas raras, jeje..



Ya no vemos nada de EVALUATION, TRIAL ni cosas de esas, además en el About podemos ver nuestro nombre gracias a los cambios que antes hicimos.

Esta parte ha sido corta pero no había mucho que contar, además no he tenido la oportunidad de probar el programa, sé que no expira porque el conteo lo llevaba el packer y tampoco se ven referencias tipo "Trial" ni cosas de esas, lo que no sé si insertará una especie de watermark en los informes creados, si alguien tiene la oportunidad de utilizarlo y encuentra algún fallo puede ponerse en contacto conmigo para estudiarlo XDD

Un saludo a TODO el Grupo, en particular a Ricardo, solid, marciano, Absolom1, RedH@wk, GUAN, k0rt, Morales, Juan José, +NCR, Carlos, Shadow&Dark, Emadicius, Yllera, ZettK, kaos_xlro, OtupAtpaxa, Ulaterk, Alfonso_iat, Mair Jr. Soft, etc, y siento de veras si momentáneamente me olvidé de alguno, siempre me pasa pero de todas maneras un saludo igual para TODOS XDD!

Y para cualquier cosa ya sabéis donde contactar conmigo: syxe05@gmail.com

NOTA: Siento de veras haber desaparecido este tiempo (por motivos ajenos al cracking por su puesto que ya os contaré) y aseguro que no volverá a suceder. Un abrazo para todos los CracksLatinoS!